

Documentacion de Procesos del Proyecto

Portada

- Titulo: Modernizacion DevOps de un Generador de Avatares Open Source
- Tipo de documento: Informe tecnico de entrega
- Autor: Antonio Fedele
- Proyecto: Avatares DevOps
- Fecha: 06-07-2026
- Repositorio: <https://github.com/afedele-devops/avatares-devops.git>

Resumen ejecutivo

Este documento consolida los procesos tecnicos y operativos del proyecto, basado en una adaptacion de un generador de avatares open source y extendido con practicas DevOps. Se describen la gobernanza del repositorio, el flujo CI/CD, la orquestacion en Kubernetes, la infraestructura como codigo y la capa de observabilidad. Tambien se incluye la trazabilidad de evidencias y el estado actual de cumplimiento para entrega.

Indice

1. Objetivo
2. Alcance
3. Metodologia de trabajo
4. Proceso de gobernanza del repositorio
5. Proceso CI/CD
6. Proceso de orquestacion en Kubernetes
7. Proceso IaC con Terraform
8. Proceso de observabilidad
9. Estado de cumplimiento y evidencias
10. Conclusiones
11. Anexos

1. Objetivo

Documentar de forma clara, verificable y exportable a PDF los procesos del proyecto para su evaluacion academica y trazabilidad tecnica.

2. Alcance

Este informe cubre:

- Gobernanza del codigo fuente y colaboracion.
- Integracion continua y despliegue continuo.
- Despliegue y operacion de la aplicacion en Kubernetes.
- Gestion de infraestructura con Terraform.
- Observabilidad con Prometheus y Grafana.
- Evidencias de validacion tecnica.

3. Metodologia de trabajo

Se trabajo con enfoque incremental por hitos:

- Definicion de normas de repositorio y flujo de ramas.
- Implementacion de pipeline CI/CD con pruebas y despliegue.
- Construccion de manifests Kubernetes y validacion funcional.

- Parametrización de infraestructura por entornos con Terraform.
- Integración de observabilidad y documentación de evidencias.

4. Proceso de gobernanza del repositorio

4.1 Estrategia de ramas

Se adopto el flujo:

- main como rama principal de entrega.
- develop como rama de integración.
- feature/* para desarrollo de funcionalidades.

4.2 Estándares de colaboración

- Convención de commits descriptivos.
- Pull Request con plantilla y checklist de revisión.
- Plantillas de issue para bug y feature request.

4.3 Referencias

- docs/process/README.md
- docs/process/branching-strategy.md
- docs/process/commit-conventions.md
- .github/pull_request_template.md
- .github/ISSUE_TEMPLATE/bug_report.md
- .github/ISSUE_TEMPLATE/feature_request.md

5. Proceso CI/CD

5.1 Workflow principal

Pipeline implementado en:

- .github/workflows/ci-cd.yml

5.2 Secuencia operativa

1. Backend tests.
 - Creación de entorno virtual.
 - Instalación de dependencias.
 - Ejecución de pruebas API.
2. Frontend tests.
 - Instalación de dependencias con npm ci.
 - Ejecución de pruebas frontend.
3. Build Docker images.
 - Build de imagen API.
 - Build de imagen WEB.
4. Publish images to registry.
 - Publicación de imágenes en GHCR.
5. Deploy to staging.
 - Aplicación de manifests en namespace de staging.

- 6. Smoke test post deploy.
- Validacion de endpoints de salud y endpoint principal.

5.3 Estado verificado

- Run verificado: 28770436928.
- Resultado final: success.
- Jobs criticos (publish, deploy, smoke): success.

6. Proceso de orquestacion en Kubernetes

6.1 Componentes de despliegue

Manifests base en k8s/:

- Deployment api.
- Deployment web.
- Service api.
- Service web.
- Ingress.
- PVC.
- ConfigMap.
- Secret.
- Kustomization.

6.2 Criterios tecnicos aplicados

- API con 1 replica por uso de SQLite.
- Probes de readiness y liveness en API y WEB.
- Resolucion interna del servicio API desde WEB.
- imagePullSecrets para imagenes privadas.
- securityContext en API para permisos de volumen.

6.3 Estado funcional

- API y WEB en estado Running.
- Ingress operativo con acceso externo.
- Endpoint /health responde 200.
- Endpoint /api/avatar/spec responde 200.

7. Proceso IaC con Terraform

7.1 Estructura por entornos

- infra/modules para modulos reutilizables.
- infra/envs/staging para entorno de pruebas.
- infra/envs/production para entorno productivo.

7.2 Backend remoto y locking

- Estado remoto en S3.
- Bloqueo de estado con DynamoDB.
- Parametros del backend inyectados con terraform init -backend-config.

7.3 Validaciones ejecutadas

- terraform init -backend=false y terraform validate en staging.
- terraform init -backend=false y terraform validate en production.

8. Proceso de observabilidad

8.1 Componentes desplegados

Overlay en k8s/observability con:

- Prometheus.
- Grafana.
- Configuración de scrape por annotations.
- Provisioning de datasource Prometheus.
- Provisioning de dashboard avatars desde ConfigMap.

8.2 Scraping de metricas

API expone annotations:

- prometheus.io/scrape=true.
- prometheus.io/path=/metrics.
- prometheus.io/port=5000.

Prometheus descubre y scrapea pods con `kubernetes_sd_configs` y `relabel_configs`.

8.3 Dashboard de Grafana

Grafana carga el dashboard de avatars desde ConfigMap por provisioning.

8.4 Estado

- Despliegue de Prometheus y Grafana: completado.
- Scraping por annotations en API: completado en manifests.
- Dashboard avatars por ConfigMap: completado en manifests.
- Evidencia visual final de targets UP y dashboard con metricas reales: pendiente de capturas consolidadas.

9. Estado de cumplimiento y evidencias

9.1 Fuentes de evidencia

- docs/evidencias/k8s/README.md
- docs/evidencias/k8s/staging-url-candidate.txt
- docs/evidencias/iac/README.md
- docs/evidencias/iac/terraform-fmt.txt
- docs/evidencias/iac/terraform-validate-staging.txt
- docs/evidencias/iac/terraform-validate-production.txt

10. Conclusiones

El proyecto presenta una implementación sólida de prácticas DevOps sobre una base open source, con CI/CD funcional, despliegue en Kubernetes e infraestructura declarativa por entornos. La trazabilidad técnica está documentada y validada. Para cierre académico, el foco restante es consolidar evidencia visual final en anexos.

11. Anexos

Anexo A. Referencias internas del proyecto

- docs/process/README.md
- docs/evidencias/k8s/README.md
- docs/evidencias/iac/README.md

Anexo B. Evidencias tecnicas graaficas anexadas

- Captura de run exitoso en GitHub Actions.
- Captura de kubectl get pods, get svc, get ingress, get pvc.
- Captura de acceso web en staging-url-candidate.
- Captura de Prometheus targets.
- Captura de dashboard avatars en Grafana.

Enlaces a evidencias de comandos kubectl en staging:

- docs/evidencias/k8s/get-pods-staging.txt
- docs/evidencias/k8s/get-svc-staging.txt
- docs/evidencias/k8s/get-ingress-staging.txt
- docs/evidencias/k8s/get-pvc-staging.txt